学习情境 L04：K8S 平台底座—从 yaml 到就绪探针（6 学时）

一、学习情境基本信息

学习领域	LF	讲次	学时	布鲁姆主层级	双元场地
LF2 平台基础设施搭建与运维	LF2（第 1/6 讲）	L04	6（校内 4 + 企业站 2）	应用（用），分析渗透	校内实训机房（kubectl 终端）+企业实训站（dap-v2 集群）

- **企业站 PR 任务**：在课程 namespace `dap-course-g0X` 内部署一套最小 dap-gateway（以 `deploy/k8s/04-gateway.yaml` 为基准裁剪），通过 8 项验收测试后将部署 yaml 以 PR 回写课程仓库（GitOps：live 变更必回写）。
- **课后费曼任务**：复述"就绪探针为什么不能只查进程存活"——向后端同学讲清 readiness 与 liveness 的分工，以及"用 liveness 查依赖"为何会 CrashLoop。

二、任务描述

某零售集团数据部收到运维部通报：dap-v2 网关上周滚动更新出现短时 502，根因是该组件 readinessProbe 缺失且未配 preStop，旧 Pod 还在接流量就被杀。运维部要求数据部按平台规范重写 dap-gateway 部署清单（Deployment/Service/ConfigMap/Secret/Ingress/探针齐全），先在教学 namespace 验证再上生产，并附一次滚动更新不中断的证明。

三、学习目标

1. 【应用】能独立编写 dap-gateway 最小部署清单（以 `deploy/k8s/04-gateway.yaml` 为基准：Deployment + Service + ConfigMap + Secret + Ingress + 三类探针 + resources），在 `dap-course-g0X` 内 `apply` 成功且 Ingress 路由可达。
2. 【应用】能正确区分并配置 livenessProbe 与 readinessProbe（/healthz、/readyz），并解释设计差异：readyz 检查关键依赖（Redis/DB PING，失败摘流量返 503），healthz 只查进程存活（失败才重启）。
3. 【分析·渗透】能通过"删 Pod / 断依赖 / 坏 Secret"三组故障实验预测并解释集群行为（自愈重建、Endpoints 摘除、挂载失败），用 `kubectl describe`/`kubectl get events` 定位原因。
4. 【应用·渗透】能完成一次安全滚动更新：改镜像 tag→`rollout status` 观察→`rollout undo` 回滚，并用压测/循环 curl 证明业务不中断。
5. 【素养·渗透】践行平台公约：Secret 不入 Git（占位 + CI 注入）、live patch 必回写仓库 yaml、所有验证以命令输出为证（无产物不声称完成）。

四、教学重点与难点

教学重点

1. 五类核心对象的职责与接线关系：Deployment（副本与滚动）、Service + Endpoints（稳定访问入口）、ConfigMap/Secret（配置与敏感配置分离）、Ingress（七层路由与 TLS）。

2. 探针三态与优雅关停：startup/liveness/readiness 分工；`terminationGracePeriodSeconds` + preStop 保证在途请求排空——这正是本次 502 事故的直接解药。

教学难点（附突破策略）

1. **readyz vs healthz 的设计辨析**——学生极易把两个探针都指到"查依赖"或都指到"查进程"。

突破策略：双实验对照法——①正确配置：kill 课程 Redis 依赖→readyz 503→Pod 从 Endpoints 摘除→上游拿到 503 优雅降级、进程不重启；②反面实验：故意把 livenessProbe 指向 /readyz→Redis 抖动→Pod 被 kubelet 反复重启→观察 CrashLoopBackOff。一正一反，肌肉记忆。

2. **Service/Endpoints 与 Ingress 的路由映射**——yaml 都能 apply，但"流量到底走没走通"心里没图。

突破策略：链路追踪法——`curl -H "Host: gw-g01.dap-course.local" http://10.167.2.175/api/ping` 起，沿 Ingress 规则→Service selector→Endpoints 列表→Pod IP 四跳逐一取证据，画成流量追踪图。

五、教学准备

类别	准备内容
环境	① 教学集群：两节点 K8S（10.167.2.175 master 16C/126G + 10.167.2.176 worker 64C/125G）；② 每组独立 namespace：`dap-course-g01~dap-course-g08`（预先建好，含 ResourceQuota）；③ 课程镜像 `registry.local/dap/gateway:course-v1.0` 与 `:course-v1.1`（内置 /healthz /readyz /api/ping，readyz 依赖同 ns 的 redis:7-alpine 课程实例）；④ Ingress-NGINX 类 `nginx`，课程域名规则 `gw-g0X.dap-course.local`
资料	基准清单 `deploy/k8s/04-gateway.yaml` 及其"填空版"（抽走探针/资源限额/env 注入/Ingress 路径/selector 五处）；《平台部署公约》（GitOps 回写/Secret 约定/验证留证）；8 项验收测试单；故障实验记录单
分组	沿用虚拟数据部；本讲主角色 = 数据工程（部署/验证），业务分析负责验收单执行与留证，质量治理负责反例实验与 events 归档
仓库	`git checkout course/L04-baseline`；PR 目标分支 `course/L04-review`
安全须知	课程 ns 已配 ResourceQuota 与默认 NetworkPolicy；严

	禁操作 dap-v2 生产 ns；真实 Secret 不落本地磁盘、不入 Git（占位 + CI 注入）

六、教学过程设计（六步法，校内 4 学时）

6.0 教学过程总表

步骤	时长	核心内容	主要产出
资讯	30′	502 事故通报精读；K8S 对象与探针原理速览	事故根因笔记
计划	30′	解读基准清单 04-gateway.yaml；定填空分工与验证顺序	部署实施计划
决策	20′	定探针参数（路径/周期/阈值）与回滚预案	参数决策卡
实施	90′	①填空编写清单 ②部署与四跳验证 ③三组故障实验 ④滚动更新与回滚	可用部署 + 实验记录
检查	40′	8 项验收测试单逐项过检 + 教师抽问	验收单签名
评价 + 学生讲堂	30′	"一次 502 事故复盘 3 分钟" + 量表评分	评价量表

6.1 资讯（30′）

- 【教师活动】投影运维部 502 事故通报（时间线：滚动更新→旧 Pod 终止→Ingress 仍指向已死 Endpoints→上游 502→3 分钟后自愈）；抛出问题："如果当时有 readinessProbe + preStop，时间线会在哪一步被改写？"随后用 10 分钟速览五类对象与三类探针的职责矩阵。
- 【学生活动】在时间线图上标注"缺失的控制点"，写出自己假设的修复方案（不许上网抄配置，先凭理解写）。
- 【教学方法】事故案例导入 + 假设先行。
- 【设计意图】从真实故障倒推知识点，让每一段 yaml 都有"它是谁的解药"的语境。

五类对象速查表（对照 `deploy/k8s/04-gateway.yaml` 段落定位）

对象	职责	在 04-gateway.yaml 中的位置	出问题的典型症状
Deployment	副本数、滚动策略、探针挂载点	apps/v1 Deployment 段	副本漂移、更新即 502
Service	稳定虚拟 IP + 按 selector 选中 Pod	Service（port 8000）段	Endpoints 为空
ConfigMap	非敏感配置注入	dap-gateway-config	改配置不重启不生效

Secret	敏感配置（密码/DSN）	dap-gateway-secret	挂载失败 Pod 起不来
Ingress	七层 host/path 路由 + TLS	networking.k8s.io Ingress 段	域名 404 / 502

6.2 计划（30′）

- 【教师活动】带读基准清单 `deploy/k8s/04-gateway.yaml`（生产版），逐段对应五类对象；发放"填空版"（抽走 5 处关键配置），说明验证顺序约束：先 apply 通过→再路由可达→再探针实验。
- 【学生活动】组内分工：谁补探针段、谁补 Ingress 段、谁准备验证命令清单；登记每个填空处"预计的作用"。
- 【教学方法】基准对照 + 分工计划。
- 【设计意图】以生产清单为锚，填空而非从零默写——降低认知负荷，保留关键决策点训练。

6.3 决策（20′）

- 【教师活动】主持探针参数决策：readiness `periodSeconds=5, failureThreshold=2`（快速摘流）vs liveness `periodSeconds=10, failureThreshold=3`（慢重启防抖）——为什么 liveness 必须比 readiness"钝"？
- 【学生活动】产出参数决策卡（每参数一行"值 + 理由"），并写回滚预案（更新出问题先 `rollout undo` 还是先摘流量？）。
- 【教学方法】参数听证。
- 【设计意图】探针参数是最易被复制粘贴略过的决策点，强制给理由。

《参数决策卡》模板（实施前定稿，验收时教师对照追问）：

参数	本组取值	理由（一句话）
readinessProbe periodSeconds		
readinessProbe failureThreshold		
livenessProbe periodSeconds		
livenessProbe failureThreshold		
terminationGracePeriodSeconds		

6.4 实施（90′）

子步骤 1：填空编写清单（20′）

在填空版基础上补全（关键段示例，其余沿用基准）：

```
# deploy/k8s/04-gateway.yaml（课程填空版·关键段）
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dap-gateway
  namespace: dap-course-g01          # 本组 ns
spec:
```

```
  replicas: 2
  template:
    spec:
      containers:
        - name: gateway
          image: registry.local/dap/gateway:course-v1.0
          envFrom:
            - configMapRef: { name: dap-gateway-config }
            - secretRef:    { name: dap-gateway-secret }
          readinessProbe:                # 摘流量用：查依赖自愈
            httpGet: { path: /readyz,  port: 8000 }
            periodSeconds: 5
            failureThreshold: 2
          livenessProbe:                 # 重启用：只查进程存活
            httpGet: { path: /healthz, port: 8000 }
            periodSeconds: 10
            failureThreshold: 3
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: dap-gateway
  namespace: dap-course-g01
spec:
  ingressClassName: nginx
  rules:
    - host: gw-g01.dap-course.local     # 本组域名
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: dap-gateway, port: { number: 8000 } } }
```

Secret 只提交占位文件 `dap-gateway-secret.yaml.example`（真实 Secret 由企业导师现场发放/CI 注入，不入 Git）。

子步骤 2：部署与四跳验证（25'）

```
kubectl -n dap-course-g01 apply -f dap-gateway-lab.yaml
kubectl -n dap-course-g01 get pods -o wide                 # 2/2 Running
kubectl -n dap-course-g01 describe pod -l app=dap-gateway  # 看 Events 与探针配置
kubectl -n dap-course-g01 get endpoints dap-gateway        # 第3跳：Endpoints 已含 2 个 PodIP
curl -s -H "Host: gw-g01.dap-course.local" http://10.167.2.175/healthz   # 200
curl -s -H "Host: gw-g01.dap-course.local" http://10.167.2.175/api/ping  # {"pong":true}
```

四跳取证：Ingress 规则（host/path）→Service selector 与 port→Endpoints PodIP 列表→Pod 内进程，把每跳证据截图贴入流量追踪图。

任一验收不过时的排障速查（按序执行，禁止无脑删了重建）：

```
kubectl -n dap-course-g01 describe pod -l app=dap-gateway   # 先看 Events：镜像拉取？探针失败？挂载失败？
kubectl -n dap-course-g01 logs deploy/dap-gateway --tail=50  # 再看应用日志：配置缺失？依赖连不上？
kubectl -n dap-course-g01 get ingress dap-gateway -o yaml    # 再核路由：host/path/backend 是否匹配
```

子步骤 3：三组故障实验（25'）

实验	操作	预期现象（须截图 events/endpoints 佐证）
自愈	`kubectl -n dap-course-g01 delete pod -l app=dap-gateway`（删 1 个）	ReplicaSet 秒级重建新 Pod；重建期间 Endpoints 只剩 1 个 IP，服务经另一副本不中断
依赖故障	停掉课程 Redis（`kubectl -n dap-course-g01 scale deploy/dap-`	/readyz 变 503→Pod 从 Endpoints 摘除→外部拿到的是 503 降级而非

	lab-redis --replicas=0）	502 连接错误；/healthz 仍 200；Pod 不重启
反例：liveness 误查依赖	临时把 livenessProbe 也指向 /readyz 后 apply	Redis 停止期间 kubelet 反复重启 Pod→CrashLoopBackOff——用事实记住"liveness 只查进程"

实验后恢复 Redis 副本，确认 Pod 自动回到 Endpoints（依赖自愈闭环）。

子步骤 4：滚动更新与回滚（20′）

```
# 循环探测放后台：watch curl -s -H "Host: gw-g01.dap-course.local" http://10.167.2.175/api/ping
kubectl -n dap-course-g01 set image deploy/dap-gateway gateway=registry.local/dap/gateway:course-v1.1
kubectl -n dap-course-g01 rollout status deploy/dap-gateway     # 逐个替换
kubectl -n dap-course-g01 rollout history deploy/dap-gateway
kubectl -n dap-course-g01 rollout undo deploy/dap-gateway       # 回滚演练
```

记录：滚动期间循环 curl 的成功率（应为 100% 或仅个别超时）、Surge/Unavailable 行为；对照事故通报说明"为什么现在的流程不会再 502"。

6.5 检查（40′）

- 【教师活动】发放 8 项验收测试单（见下表）逐组现场过检，随机抽问"这个检查项防的是哪类事故"。
- 【学生活动】逐项执行留证（命令＋输出截图）；未过项当场修复复测；验收单双签（组内自检＋教师复检）。
- 【教学方法】验收测试单（清单式过检）。
- 【设计意图】把"部署成功"定义成可复核的 8 个事实，而非"没有报错"。

8 项验收测试单（逐项留证：命令＋输出截图）

#	验收项	命令/方法	通过标准
1	副本就绪	kubectl get pods	2/2 Running
2	Endpoints 就绪	kubectl get endpoints dap-gateway	含 2 个 PodIP
3	进程存活	curl …/healthz	HTTP 200
4	依赖就绪	curl …/readyz	HTTP 200
5	外部路由	curl -H Host …/api/ping	返回 pong
6	删 Pod 自愈	delete pod 后计时	≤60s 恢复且服务不断
7	依赖故障降级	redis 副本置 0	外部得 503 且 Pod 不重启
8	滚动零中断＋可回滚	set image＋undo	循环 curl 无失败

6.6 评价＋学生讲堂（30′）

- 【教师活动】抽 2 组各 3 分钟讲堂："给运维部的一次 502 事故复盘"（时间线＋根因＋三层修复：探针/preStop/回滚预案）；组织量表评分；总结并布置企业站任务（回写 PR）与费曼任务。

- 【学生活动】讲堂复盘（必须引用本组实验数据）；自评互评；登记回炉点。
- 【教学方法】费曼讲台 + 事故复盘写作框架（时间线-根因-修复-预防）。
- 【设计意图】复盘框架是运维岗位核心可迁移技能；实验数据引用延续"用证据说话"课程主线。

教师收束两句话：①探针不是"配了就行"，参数本身就是你的故障处理策略；②今天的 8 项验收单，就是你以后每个服务上线检查单的雏形——把它带进 L05 基建部署。

七、企业站实践任务卡（2 学时）

项目	内容
任务目标	在课程 namespace 完成最小 dap-gateway 部署并通过验收，且按 GitOps 公约回写仓库
操作步骤	① 在企业导师处领取本组真实 Secret（不落盘，CI 注入）；② 基于校内清单在 `dap-course-g0X` 完整部署并通过 8 项验收；③ `git checkout -b g0X/L04-gateway` 基于 course/L04-baseline，将最终 yaml 放入 `deploy/k8s/lab/g0X/`（Secret 仅提交 .example 占位）；④ PR 至 course/L04-review，描述中逐项列出 8 项验收证据链接/截图；⑤ 回应企业导师 review（重点查：探针参数理由、Secret 是否残留）
交付物	PR 链接；8 项验收证据包；集群内存活的 dap-gateway（Ingress 域名 gw-g0X.dap-course.local 可访问）
评审标准	8 项验收全绿（4）；yaml 与集群实际一致、Secret 无泄露（3）；探针参数附理由（2）；review 往复与修复质量（1）

双导师签栏	签名	日期
企业导师		
校内导师		

八、评价设计（任务单评分表）

评价维度	权重	评分标准
部署清单质量	20	优：五类对象齐全 + 探针/资源/注入无缺漏 + 参数有理由；良：缺 ≤1 处非关键；中：缺关键探针；差：无法 apply
四跳流量取证	15	优：四跳证据齐全成图；良：缺 1 跳；中：仅 curl 通；差：未取证
故障实验记录	25	优：三组实验现象与解释全对（含反例 CrashLoop 归因）；良：对 2

		组；中：做了没解释；差：≤1 组
滚动更新与回滚	15	优：零中断数据 + history/undo 证据；良：有中断但归因清楚；中：只执行未记录；差：未做
验收单与讲堂	25	8 项全绿 15 分（每项 1.875，部分过按比例）；复盘讲堂 10 分（时间线-根因-修复-预防四段齐）

过程性观察点：①决策环节学生是否把 liveness 阈值设得比 readiness 更保守（概念内化信号）；②反例实验看到 CrashLoop 时是先删配置恢复还是先截图留证（运维职业习惯）；③是否有组尝试把真实 Secret 提交进 Git（当场拦截并回炉公约）。

九、课后费曼任务

项目	内容
讲给谁	一位后端/测试方向的同学（不熟 K8S 者最佳）
讲什么	"就绪探针为什么不能只查进程存活"：用本组实验讲清三个事实——①进程活着≠能干活（Redis 断了 /healthz 仍是 200）；② readiness 摘流量、liveness 才重启，分工不能互换；③把 liveness 指向 /readyz 会让依赖抖动变成重启风暴（CrashLoopBackOff 实录）。允许画" kubelet - Pod - Endpoints - 流量"四格图
自评三问	① 我能只用"门口挂牌"这类生活比喻讲清两种探针分工吗？② 我卡壳在哪——Endpoints 摘除机制还是优雅关停时序？③ 卡壳处回炉：本讲故障实验记录单 + 《平台部署公约》探针章节；仍不通则下次课企业站请教导师实操演示

讲述提纲提示：先用"进程活着≠能干活"一句话立论→再讲本组 Redis 停止实验的两个截图（readyz 503 摘流量 / healthz 200 不重启）→ 最后讲反例 CrashLoop 的因果链收尾。

十、教学反思栏（授课后填写）

反思维度	记录
目标达成分度（5 个目标逐条）	
学生卡点（填空段？反例实验？Ingress 追踪？）	
时间偏差（故障实验是否挤压滚动更新）	
材料与改进（填空版粒度/镜像内置行为）	
需在后续讲次回炉的知识点	
优秀表现 / 学生金句记录	
需个别辅导学生与安排	